

Índice

1. [Introducción](#)
 2. [Documentación de usuario](#)
 - [Descripción funcional](#)
 - [Tecnologías](#)
 - [Instalación](#)
 - [Acceso al sistema](#)
 - [Manual de referencia](#)
 3. [Documentación del sistema](#)
 - [Especificación del sistema](#)
 - [Módulos](#)
 - [Plan de pruebas](#)
-

Documentación creada por Cecilia Gallardo Acevedo y revisada por Susana Fernandez Sierra.
Pruebas realizadas y escritas respectivamente por cada uno.

Introducción

En este documento documentaremos la información necesaria para el uso del usuario y entendimiento del código hecho por los alumnos, además de sus pruebas para la demostración del correcto funcionamiento. El programa en concreto consiste en la implementación de un juego de aventura, concretamente un escape room, en donde el jugador en base a objetos y puzles irá abriendo conexiones a las salas hasta llegar a la salida.

Documentación de usuario

Para el uso del programa el usuario necesitara algún programa el usuario debera tener todos los ficheros .c, .h y .txt que componen el programa en su totalidad. Además de esto el usuario necesitara un software capaz de compilar y ejecutar el programa, como por ejemplo Visual Studio Code. Trás compilar y ejecutar el código del programa el usuario ya tendrá acceso a todas las funciones para

la utilización del programa que mediante elecciones pedidas por pantalla podrá acceder a todas las funciones que implementan la funcionalidad del escape room.

Descripción funcional

El sistema, como proposito tiene la intención de implementar un escape room para el usuario.

El usuario mediante el programa tendrá la opción de crear o cargar una nueva partida. Si selecciona la nueva partida, se pedirán los datos del jugador por pantalla y comenzara el juego. Si se escoge la opción de cargar partida se utilizarán los datos de la partida guardada previamente del jugador y se comenzara en el juego. Una vez el jugador comience su aventura tendrá las siguientes opciones para el avance del juego:

- 1. Describir sala
- 2. Examinar (objetos y salidas)
- 3. Entrar en otra sala
- 4. Coger objeto
- 5. Soltar objeto
- 6. Inventario
- 7. Usar objeto
- 8. Resolver puzle / introducir código
- 9. Guardar partida
- 10. Volver

Una vez el usuario tenga intención de salir del programa podrá salirse desde el menú principal con la opción de salir.

Tecnología

Las tecnologías que usamos en la creación del programa se componen de:

- [Visual Studio Code](#): Version 1.114
- [Github](#)

Manual de instalación

Para la instalación del sistema, como anteriormente se ha mencionado, se necesitarán todos los ficheros .c, .h y punto .txt asociados al programa y un software que tenga un compilador capacitado para C y su ejecución.

Acceso al sistema

Para acceder al sistema el usuario descargara el proyecto entero y abra el programa que prefiera para su uso, tras ello solo debera ejecutar el código. Al ejecutarlo el jugador, tras que el juego le de la bienvenida, podrá escoger las opciones de juego por pantalla mediante los menús que desee.

En el primer menu, el usuario se encontrara con las siguientes opciones:

- 1. Nueva partida
- 2. Cargar partida
- 3. Salir

El cargar o comenzar una partida llevara al mismo menu (Aunque en Nueva partida sera necesario que el usuario meta sus datos y en Cargar partida necesitara verificar el usuario), mientras que Salir terminara la ejecución del programa.

El segundo menu que nos podremos encontrar sera este:

- 1. Describir sala
- 2. Examinar (objetos y salidas)
- 3. Entrar en otra sala
- 4. Coger objeto
- 5. Soltar objeto
- 6. Inventario
- 7. Usar objeto
- 8. Resolver puzle / introducir código
- 9. Guardar partida
- 10. Volver

El cual ya vimos con anterioridad y permite al usuario el acceso a la jugabilidad y al menu anterior.

Manual de referencia

Como ventaja para el usuario en el uso del programa que hemos creado, se obtendra la experiencia de un juego de aventuras tipo escape room en que el usuario podra disfrutar y obtener diversión, asi como llegar al final del juego en base a la resolución y uso de puzles por las distintas salas.

Documentación del sistema

El proyecto esta programado en su totalidad en C, se compone de una totalidad de 10 modulos (Con sus respectivos .h y .c, este ultimo siempre que sean necesarios), unos .txt que permiten tener datos para el juego y un main. El funcionamiento de la aplicación se basa en el uso de seis estructuras en las cuales almacenaremos los datos (los cuales coinciden con la información de los .txt) y estas, a su vez, se almacenan en vectores que imitan los objetos de C++. La implementación de las demas funciones y su uso se ha llevado a cabo y comentado teniendo en cuenta estas estructuras y "objetos". Para el

mantenimiento orientado a futuro se han llevado a cabo directamente varios comentarios en el código fuente y varias cabeceras que explican con claridad que hace cada parte del código.

Especificación del sistema

En nuestra visión personal para la distribución de los modulos hemos tenido en cuenta tanto que queremos hacer tanto con que tenemos que hacerlo. Sabemos que para la composición de este proyecto hay una gestión de ficheros, unos menús por los cuales podremos navegar en el juego, unas mecanicas que nos haran poder jugar al juego y la necesidad de trabajar con estructura (y sus correspondientes vectores dinamicos) para la facilitación del trabajo. Teniendo en cuenta esto, desglosamos el trabajo en diez modulos, el listado de todos ellos seria el siguiente:

- 1. Partida
- 2. Salas
- 3. Conexión
- 4. Jugador
- 5. Objetos
- 6. Puzzles
- 7. Juego
- 8. Ficheros
- 9. Lógica
- 10. Menú

Esta sección debe describir el análisis y la especificación de requisitos. Cómo se descompone el problema en distintos subproblemas y los módulos asociados a cada uno de ellos, acompañados de su especificación. También debe incluir el plan de desarrollo del *software*.

Módulos

Tanto en partida, salas, conexión, jugador, objetos, puzzles encontraremos las estructuras en las cuales almacenaremos las variables y datos que necesitamos durante todo el código para la creación de las funciones que los componen. En juego, siguiendo con la necesidad del uso de la estructura, al disponer de varios datos en los mismos ficheros que usaran a estas, se hace el manejo de los vectores que almacenaran a estas estructuras. En ficheros tendremos todas las funciones que hara todas las actualizaciones necesarias a estos, así como todo el vertido de estos a las estructuras y vectores. En lógica se podrá encontrar todo lo necesario para el desarrollo de este, es decir funciones como coger objetos, soltarlos, etc. Finalmente, en menú podremos encontrar todos los menus que nos permitan seleccionar que acción queremos hacer dentro del juego.

Plan de prueba

Prueba de los módulos

Debido a la gran cantidad de pruebas realizadas en todo el código, colocaremos una pequeña cantidad como ejemplos, ya que todo el trabajo ha sido probado de la misma manera, y con ello aseguramos la rigurosidad de las pruebas sin la necesidad de tener que meter tal cantidad exuberante de pruebas.

Prueba de caja blanca

Ejemplo 1 Cecilia:

De la función void nuevapartida(jugador *j, partida *p, int *jug) analizare esta parte de petición de nombre usuario

```
do{
    printf("Introduce tu nombre de usuario: \n");
    scanf("%s", j[*njug-1].jugador);

    do{
        for (y=0; y<njug; y++){
            if(strcmp(j[*njug-1].jugador, j[y].jugador)==0){          //Comprobamos que el nomb
                printf("El nombre de usuario ya existe, introduce otro: \n");
                scanf("%s", j[*njug-1].jugador);
            }
        }
    }while(strcmp(j[*njug-1].jugador, j[y].jugador)==0);

    do{
        printf("El nombre de usuario introducido es: %s, ¿es correcto? Si:1 No:0 \n", j[*njug-1].jugador);
        scanf("%d", &Correcto);
    }while (Correcto!=1 && Correcto!=0);

    while(Correcto==0);
}
```

El fragmento de código presenta una estructura compuesta por varios bucles do while anidados, junto con un bucle for y una condición if.

El bucle for recorre el vector de jugadores desde la posición inicial hasta el número total de jugadores, permitiendo comparar el nombre introducido con los ya existentes. En caso de coincidencia, la condición if detecta que el nombre está repetido y solicita al usuario la introducción de uno nuevo.

El do while interno garantiza que el proceso se repita hasta que el nombre introducido no coincida con ninguno de los existentes en el sistema.

Posteriormente, otro do while se encarga de solicitar la confirmación del usuario, asegurando que el valor introducido sea válido (0 o 1).

Finalmente, el do while externo controla que, en caso de que el usuario no confirme el nombre (valor 0), se repita todo el proceso desde el inicio.

Pruebas de ruta básica

Ruta básica Cecilia:

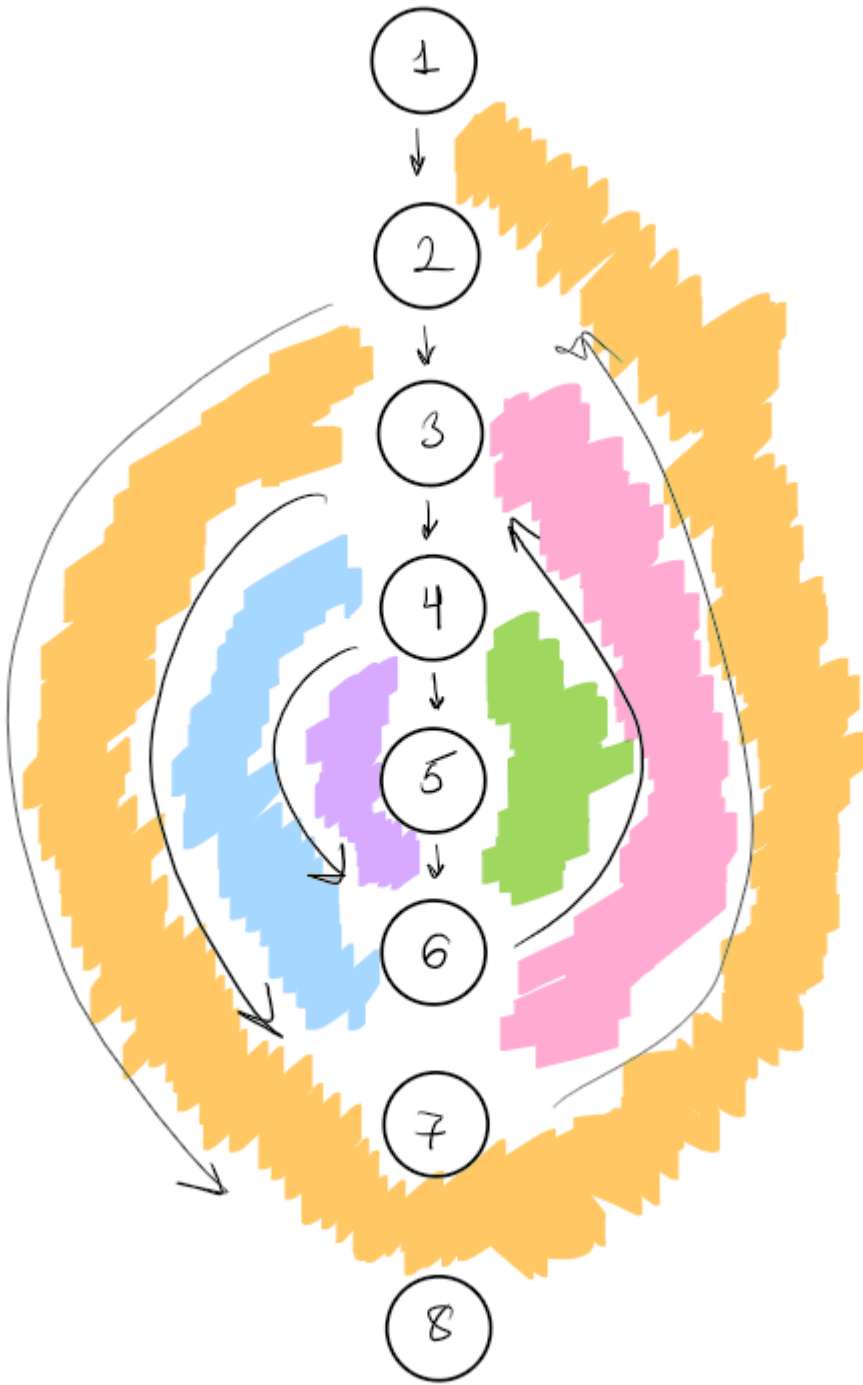
```
//Cabecera: void mostrarinventario(jugadores *j, objetos *o, partida p, int *numobj, int *jug)
//Precondición: La funcion recibe la estructura de jugadores, objetos y partida inicializada
//Postcondición: El jugador mostrara los objetos que tiene en su inventario
void mostrarinventario(jugador *j, objetos *o, partidas p,int *numobj, int * jug){
    int x,y;
    printf("Los objetos en tu inventario son:\n");
    for(x=0; x<=j[*jug].cant_obj; x++){
        for(y=0; y<=*numobj; y++){           //Recorremos el vector del inventario y lo mostramos
            if(strcmp(j[*jug].inv[y].objinv,o[x].id_obj)==0){
                printf("%d %s %s %s %s \n", x, o[x].id_obj, o[x].nomb_obj, o[x].desc, o[x].localiz);
            }
        }
    }
}
```

La función a la que se le hare las pruebas de caja blanca, específicamente, caja blanca será esta:

Pasada a pseudocódigo para su análisis la función quedaría así:

```
...
void: función mostrarinventario(E jugadores: j, E objetos: o, E partidas: p, E entero: jug)
var
inicio
entero: x, y
    escribir "Los objetos en tu inventario son:" (1)
    desde x ← 0 hasta j[jug].cant_obj (2)
        desde y ← 0 hasta numobj(3)
            si j[jug].inv[y].objinv = o[x].id_obj entonces (4)
                escribir x, o[x].id_obj, o[x].nomb_obj, o[x].desc, o[x].localiz (5)
            fin_si (6)
        fin_desde (7)
    fin_desde (8)
fin_función
...
```

Control de flujo:



Complejidad ciclomática:

$$V(G) = NA - NN + 2 = 10 - 8 + 2 = 2 + 2 = 4$$

$$V(G) = NNP + 1 = 3 + 1 = 4$$

$$V(G) = \text{número de regiones} (R1 + R2 + R3 + R4) = 4$$

Rutas básicas linealmente independientes:

Ruta 1: 1-2-8

Ruta 2: 1-2-3-7-2-8

Ruta 3: 1-2-3-4-6-3-7-2-8

Ruta 4: 1-2-3-4-5-6-3-7-2-8

Ruta básica Mario:

Para el numero de rutas, necesitamos calcular la complejidad ciclomatica. Para ello, contamos los nodos predicados (if, else, etc) y le sumamos uno. En este caso nos sale un total de 11 rutas

- Ruta 1: Falla la apertura de "Jugadores.txt" (`f_jug == NULL` es Verdadero) y no hay ninguna partida en memoria (`p_activa != NULL` es Falso). El programa imprime el error de escritura por consola, salta el bloque de guardado de jugadores y luego imprime que no hay partida.
- Ruta 2: Se abre correctamente "Jugadores.txt" (`f_jug == NULL` es Falso), pero hay 0 jugadores (`total_jugadores > 0` es Falso) y no hay partida (`p_activa != NULL` es Falso). El programa no entra al bucle principal, cierra el archivo de jugadores y avisa de que no hay partida.
- Ruta 3: Hay 1 jugador (`total_jugadores > 0` es Verdadero), pero este jugador tiene 0 objetos en el inventario (`cant_obj > 0` es Falso). No hay partida (`p_activa != NULL` es Falso). El programa escribe los datos del jugador, se salta el bucle del inventario, y termina indicando que no hay partida.
- Ruta 4: Hay 1 jugador y este tiene 1 objeto (`cant_obj > 0` es Verdadero). No hay partida (`p_activa != NULL` es Falso). El programa entra a todos los bucles de la primera mitad, guarda el objeto, y finaliza sin guardar partida.
- Ruta 5: Hay una partida en curso (`p_activa != NULL` es Verdadero), pero falla la apertura de "Partida.txt" (`f_part == NULL` es Verdadero). El programa imprime el error de acceso al archivo de la partida y finaliza.
- Ruta 6: Se abre "Partida.txt" correctamente (`f_part == NULL` es Falso), pero la partida está completamente vacía de contenido: 0 objetos, 0 conexiones y 0 puzles. El programa no entra a ninguno de los tres bucles y finaliza el guardado.
- Ruta 7 (Forzar Objetos): La partida tiene 1 objeto (`num_objetospar > 0` es Verdadero), pero 0 conexiones y 0 puzles. El programa entra al primer bucle, escribe el objeto y se salta el resto.
- Ruta 8 (Forzar Conexión - Activa): La partida tiene 0 objetos, 0 puzles, pero tiene 1 conexión y su estado es activa (`activa == TRUE` es Verdadero). El programa evalúa el if interno de conexiones y asigna el texto "Activa".
- Ruta 9 (Forzar Conexión - Bloqueada): Igual que la Ruta 8, pero la condición cambia: el estado de la conexión no es activa (`activa == TRUE` es Falso). El programa entra al else interno y asigna el texto "Bloqueada".
- Ruta 10 (Forzar Puzzle - Resuelto): La partida tiene 0 objetos, 0 conexiones, pero tiene 1 puzle y está resuelto (`resuelto == TRUE` es Verdadero). El programa evalúa

el if interno de puzles y asigna el texto "Resuelto".

- Ruta 11 (Forzar Puzzle - Pendiente): Igual que la Ruta 10, pero la condición cambia: el puzzle no está resuelto (resuelto == TRUE es Falso). El programa entra al else interno de puzles y asigna el texto "Pendiente".

Ruta básica Francisco:

Esta prueba garantiza que se exploran los posibles caminos de ejecución de la implementación. Siguiendo los pasos definidos para la prueba de ruta básica:

Pasos 1 y 2: Grafo de Flujo (CFG) y Complejidad Ciclomática

La función menu_principal() está compuesta por un bucle principal do-while (1 decisión de salida), una estructura condicional if-else para validar rangos (1 decisión) y una estructura switch con 3 opciones operativas (equivale a 2 decisiones lógicas subyacentes).

- Fórmula aplicada: $V(G) = \text{Nodos Predicado} + 1 = 4 + 1 = 5$.
- La complejidad ciclomática calculada es 5.

Paso 3: Conjunto básico de rutas linealmente independientes

Se han identificado las siguientes rutas principales:

- Ruta 1 (Validación fallida): Entra al bucle -> Falla la condición del if -> Muestra error -> Repite el bucle.
- Ruta 2 (Caso 1): Entra al bucle -> Pasa el if -> Ejecuta Nueva Partida -> Repite el bucle.
- Ruta 3 (Caso 2): Entra al bucle -> Pasa el if -> Ejecuta Cargar Partida -> Repite el bucle.
- Ruta 4 (Caso 3 / Salida): Entra al bucle -> Pasa el if -> Ejecuta Salir -> Termina la condición del while y finaliza.

Paso 4: Preparación de Casos de Prueba (Prueba de Bucles) Dado que la función se basa en un bucle interactivo, se aplican criterios de prueba de bucles para forzar distintas iteraciones:

- Prueba de Ruta 1:
 - Entradas de teclado: Primero se teclea 4 y, en la siguiente iteración, se teclea 3.
 - Descripción: Fuerza el fallo en el if por un valor no válido (OPCION INCORRECTA) y luego sale del programa de forma segura.
 - Cobertura de Bucle: Prueba de dos iteraciones.
- Prueba de Ruta 2:
 - Entradas de teclado: Primero se teclea 1 y, al volver al menú, se teclea 3.
 - Descripción: Fuerza la entrada al caso de Nueva Partida y posteriormente evalúa la salida.
 - Cobertura de Bucle: Prueba de dos iteraciones.
- Prueba de Ruta 3:
 - Entradas de teclado: Primero se teclea 2 y, al volver al menú, se teclea 3.
 - Descripción: Fuerza la entrada al caso de Cargar Partida y posteriormente evalúa la salida.

- Cobertura de Bucle: Prueba de dos iteraciones.
- Prueba de Ruta 4:
 - Entrada de teclado: Se teclea directamente 3.
 - Descripción: Ejecuta la opción de salida en el primer intento.
 - Cobertura de Bucle: Prueba de una única iteración.

Ruta básica Susana:

Esta función se encuentra en el archivo lógica.c, hecho por Cecilia Gallardo y corregido por Susana Fernández:
La función permite al jugador coger objetos que estén en la sala actual. Recorre todos los objetos y, si alguno coincide con la sala donde está el jugador, le pregunta si quiere cogerlo. Si el jugador acepta y tiene espacio en el inventario, el objeto se añade al inventario. Si no hay ningún objeto en la sala, muestra un mensaje informándolo.

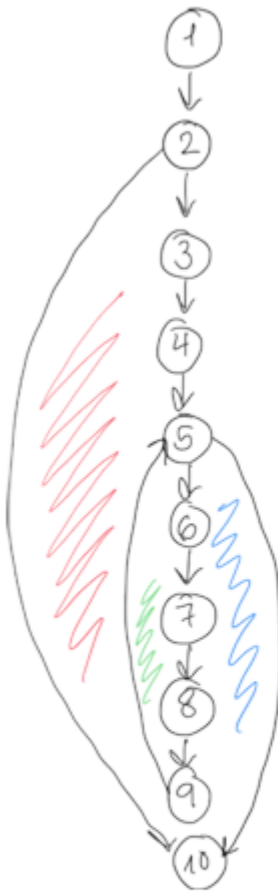
```
/* Función: void cogerobjeto(jugador* j, objetos o, partida p)
 * Descripción: La función recibe la estructura de jugador, objetos y partida
 * Modificaciones: Si jugador quiere coger objetos que estén en la sala actual, se añaden en el inventario
 */
void cogerobjeto(jugador* j, objetos o, partida p, int *numobj, int *jug, int *par)
{
    int i;
    int x;

    for(i=0; i<j[jug].cant_obj; i++)
    {
        if (strcmp(p[par].sala_actual, o[i].localiz)=0)
        {
            if (j[jug].tamainv > j[jug].cant_obj) //Si el inventario del jugador no está lleno, se le pregunta si quiere coger el
            {
                printf("Puedes coger el objeto %s, ¿quieres cogerlo? Si:1 No:0\n", o[i].nomb_obj);
                scanf("%d", &coger);
                while (coger!=1 && coger!=0)
                {
                    printf("Error\n");
                    scanf("%d", &coger);
                }
                if (coger==1)
                {
                    j[jug].inv = realloc(j[jug].inv, (j[jug].tamainv + sizeof(j[jug].inv[0])));
                    j[jug].cant_obj++;
                    strcpy(j[jug].inv[j[jug].cant_obj-1].objinv, o[i].id_obj);
                }
            }
        }
    }

    if (i==0)
    {
        printf("No hay ningún objeto en esta sala\n");
    }
}
```

```
1 función cogerobjeto(E/S j: jugador*, E o: objetos*, E p: partidas*, E numobj: entero*, E jug: entero*, E par: 1
entero*)
2 var
3 entero: coger, i, x, objetos_en_sala
4 inicio
5 i ← 0
6 objetos_en_sala ← 0
7
8 para x ← 0 hasta j[jug].cant_obj hacer 2
9 si strcmp(p[par].sala_actual, o[x].localiz) = 0 entonces 3
10 objetos_en_sala ← 1 4
11
12 si j[jug].tamainv > j[jug].cant_obj entonces 5
13 repetir 6
14 escribir("Puedes coger el objeto ", o[x].nomb_obj, ", ¿quieres cogerlo? Si:1 No:0") 7
15 leer(coger) 8
16 hasta que coger = 1 o coger = 0 9
17
18 si coger = 1 entonces
19 j[jug].tamainv ← j[jug].tamainv + 1
20 j[jug].inv ← realloc(j[jug].inv, j[jug].tamainv * sizeof(j[jug].inv[0]))
21 j[jug].cant_obj ← j[jug].cant_obj + 1
22 strcpy(j[jug].inv[j[jug].cant_obj - 1].objinv, o[x].id_obj)
23 fin_si
24 fin_si
25 fin_si
26 fin_para
27
28 si objetos_en_sala = 0 entonces
29 escribir("No hay ningún objeto en esta sala")
30 fin_si
31 fin_función 10
```

No lo incluyo como nodo



Complejidad ciclomática

1. $V(G) = NA - NN + 2$

NN: 10 nodos

NA: 11 aristas

$V(G) = 11 - 10 + 2 = 3 //$

2. $V(G) = NNP + 1$

Nodos predicado 2 y 5

$V(G) = 2 + 1 = 3 //$

3. Número de regiones

como se puede ver, hay 3.

Rutas

Ruta 1: 1-2 (0 OBJETOS DE JUGADOR)-10

Caso de prueba: El jugador tiene 0 objetos.

Ruta 2: 1-2-3-4-5(tamainv<cant_obj)-10

Caso de prueba: Tener más objetos de los que permite el inventario

Ruta 3: 1-2-3-4-5(tamainv>cant_obj)-6-7-8-9-5(tamainv<cant_obj)-10

Prueba de caja negra

Prueba de caja negra Cecilia:

```
C:\Users\HP\Desktop\Untitled x + v
=== PRUEBA DE CAJA NEGRA ===
Jugadores existentes:
- Juan
- Ana
- Luis

Introduce tu nombre de usuario:
Juan
El nombre de usuario ya existe, introduce otro:
Maria
El nombre de usuario introducido es: Maria, ¿es correcto? Si:1 No:0
5
El nombre de usuario introducido es: Maria, ¿es correcto? Si:1 No:0
0

Introduce tu nombre de usuario:
Mariana
El nombre de usuario introducido es: Mariana, ¿es correcto? Si:1 No:0
1

Usuario final registrado: Mariana

Process returned 0 (0x0)   execution time : 18.888 s
Press any key to continue.
|
```

Prueba de caja negra Mario:

Caso de Prueba 1: Límite de rango inferior (No válido). Se evaluará el comportamiento del sistema al recibir un valor sin sentido lógico para la cantidad de jugadores. Probamos el valor `total_jugadores = -1`.

- Resultado esperado: El programa no debe iterar sobre la lista de jugadores ni escribir basura en el archivo de texto. No debe producirse ningún cierre inesperado (cuelgue) del programa.

```

int main(void){

    guardar_datos(NULL,-1,NULL);

    return 0;
}

void guardar_datos(jugador *lista_jugadores, int total_jugadores, partidas *p_activa) {

    printf("\n Iniciando guardado de seguridad...\n");

    //guardar jugadores
    FILE *f_jug = fopen("Jugadores.txt", "w");
    if (f_jug == NULL) {
        printf("[ERROR] No se pudo acceder a Jugadores.txt para escritura.\n");
    } else {
        // Recorremos el array escribiendo directamente al buffer del archivo
        for (int i = 0; i < total_jugadores; i++) {
            fprintf(f_jug, "%s-%s-%s-%s",
                lista_jugadores[i].id_jugador,
                lista_jugadores[i].nomb_jugador,
                lista_jugadores[i].jugador,
                lista_jugadores[i].jugador,
            );
        }
    }
}

```

Line: 78 Col: 1

```

Iniciando guardado de seguridad...
No hay ninguna partida en curso para guardar.
Guardado finalizado.

Program ended with exit code: 0

```

Caso de Prueba 2: Límite de rango válido (Mínimo). Se comprobará la respuesta del sistema al recibir `total_jugadores = 0`.

- Resultado esperado: El archivo "Jugadores.txt" debe generarse correctamente, pero debe quedar completamente en blanco.

Caso de Prueba 3: Se verificará el correcto procesamiento de los estados booleanos internos de la partida. Se enviará una estructura `p_activa` válida donde al menos una conexión tenga su estado `activa = TRUE` y un puzle tenga `resuelto = TRUE`.

- Resultado esperado: Al abrir el archivo generado "Partida.txt", debe aparecer explícitamente escrita la palabra "Activa" asociada a la conexión, y la palabra "Resuelto" asociada al puzle.

Caso de Prueba 4: Fallo de entorno (Sistema de Archivos). Se probará la robustez del código frente a problemas externos no relacionados con la memoria del programa. Se ejecutará la función en un directorio de sistema donde se hayan retirado los permisos de escritura.

- Resultado esperado: Las funciones `fopen` devolverán `NULL`. El programa debe capturar este error, mostrar los mensajes de error y finalizar la función de forma limpia.

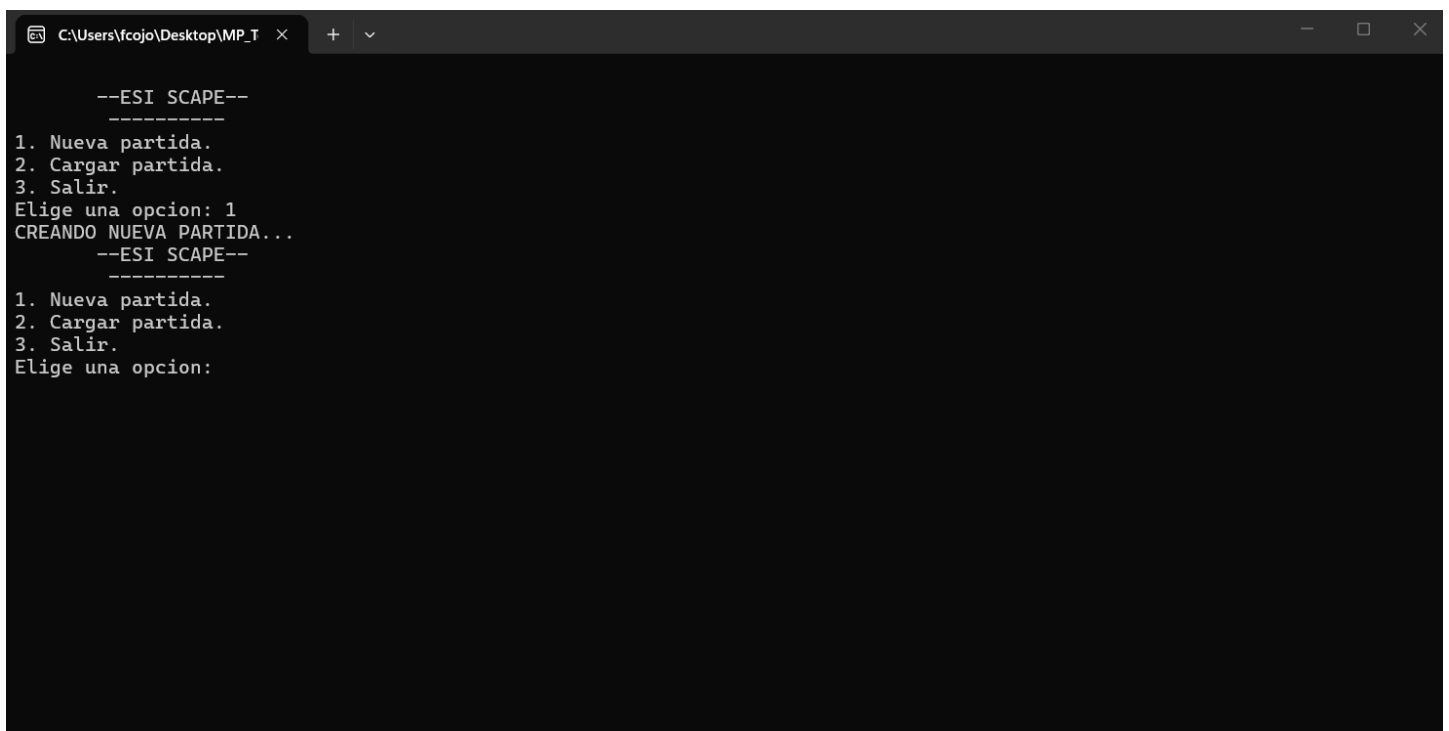
Prueba de caja negra Francisco:

Esta técnica se utiliza para determinar si la función realiza la tarea para la que ha sido creada, analizando sus entradas y comprobando que las salidas son las esperadas.

Para la función `menu_principal()`, el dato de entrada esperado es un valor numérico entre 1 y 3. Se han diseñado los casos de prueba evaluando los valores extremos y los valores justo por encima y por debajo de dichos rangos:

- **Objetivo:** Verificar que el menú principal reacciona y navega correctamente según la opción introducida por teclado.

- **CP_01 (Límite inferior válido):** Dato de Entrada: 1. Resultado Esperado: El programa ejecuta el caso 1 y llama a la función `crear_nueva_partida()`. Resultado Obtenido:



```
C:\Users\fcojo\Desktop\MP_T >
--ESI SCAPE--
-----
1. Nueva partida.
2. Cargar partida.
3. Salir.
Elige una opcion: 1
CREANDO NUEVA PARTIDA...
--ESI SCAPE--
-----
1. Nueva partida.
2. Cargar partida.
3. Salir.
Elige una opcion:
```

- **CP_02 (Límite superior válido):** Dato de Entrada: 3. Resultado Esperado: El programa muestra el mensaje de despedida y finaliza la ejecución.

- **CP_03 (Valor medio válido):** Dato de Entrada: 2. Resultado Esperado: El programa ejecuta el caso 2 y llama a la función `cargar_partida_existente()`.

- **CP_04 (Justo por debajo del límite - No válido):** Dato de Entrada: 0. Resultado Esperado: El programa detecta el error, muestra "OPCION INCORRECTA" y vuelve a solicitar entrada.

- **CP_05 (Justo por encima del límite - No válido):** Dato de Entrada: 4. Resultado Esperado: El programa detecta el error, muestra "OPCION INCORRECTA" y vuelve a solicitar entrada.

Prueba de caja negra Susana:

Función Crearsala:

```
01-Conserjería-NORMAL-Pequeño habitáculo con mostrador que se encuentra a la entrada de la ESI, antes de acceder al Hall
02-Escalera principal-NORMAL-Escalera principal de subida a la primera planta situada frente a conserjería
03-Cafetería-NORMAL-Cafetería situada en la primera planta, después de subir la escalera principal
04-Hall principal-NORMAL-Espacio amplio situado tras la entrada principal donde se distribuyen los accesos a diferentes zonas de la ESI
05-Pasillo norte-NORMAL-Pasillo situado en la planta baja que conecta varias aulas y laboratorios del ala norte
06-Pasillo sur-NORMAL-Pasillo situado en la planta baja que conecta diferentes despachos y salas del ala sur
07-Aula 1-NORMAL-Aula destinada a clases teóricas situada en la planta baja cerca del pasillo norte
08-Aula 2-NORMAL-Aula destinada a clases teóricas situada junto al aula 1 en la planta baja
09-Aula 3-NORMAL-Aula situada en la primera planta utilizada para clases y seminarios
10-Aula informática 1-NORMAL-Aula equipada con ordenadores para prácticas de programación y software
11-Aula informática 2-NORMAL-Aula con equipos informáticos destinada a prácticas de distintas asignaturas
12-Laboratorio de redes-NORMAL-Laboratorio equipado con dispositivos de red para prácticas de comunicaciones
13-Laboratorio de hardware-NORMAL-Sala equipada con material electrónico para prácticas de hardware
14-Sala de estudio 1-NORMAL-Sala tranquila destinada al estudio individual o en pequeños grupos
15-Sala de estudio 2-NORMAL-Sala destinada al estudio situada en la primera planta cerca de la biblioteca
16-Biblioteca-NORMAL-Espacio destinado a la consulta y préstamo de libros y material académico
17-Despacho profesores 1-NORMAL-Despacho utilizado por profesores para tutorías y trabajo académico
18-Despacho profesores 2-NORMAL-Despacho situado en la primera planta destinado a personal docente
19-Sala de reuniones-NORMAL-Sala destinada a reuniones del profesorado o del personal administrativo
20-Secretaría-NORMAL-Oficina donde se realizan trámites administrativos y gestión académica
21-Sala de servidores-NORMAL-Sala técnica donde se encuentran los servidores y equipos de red del edificio
22-Almacén-NORMAL-Sala utilizada para guardar material y equipamiento del centro
23-Baños planta baja-NORMAL-Baños situados en la planta baja cerca del pasillo sur
24-Baños primera planta-NORMAL-Baños situados en la primera planta cerca de la cafetería
25-Sala de descanso-NORMAL-Espacio destinado al descanso de estudiantes con mesas y asientos
26-Sala de proyectos-NORMAL-Sala utilizada por estudiantes para desarrollar trabajos y proyectos en grupo
27-Aula magna-NORMAL-Sala grande destinada a conferencias, presentaciones y eventos académicos
28-Salida de emergencia-NORMAL-Puerta señalizada utilizada como vía de evacuación en caso de emergencia
29-Escalera secundaria-NORMAL-Escalera situada en el ala sur que conecta la planta baja con la primera planta
30-Exterior-SALIDA-La salida al exterior está aquí
Número de líneas: 30
```

Prueba de integración

Como ya se dijo en las pruebas anteriores y por las mismas razones pondremos unos cuantos ejemplos de todos los códigos que componen el trabajo.

Prueba de caja blanca

La integración de todos los módulos (aunque se han ido haciendo falta entre sí para su uso entre funciones) se ha hecho finalmente en el main.c, donde se han llamado a las funciones y declarado localmente las variables que utilizaremos para la llamada de esta. Como no hacemos uso de bucles y anteriormente ya nos hemos asegurado de la rigurosidad y funcionamiento de los módulos por separado sabemos que el código usado en la integración efectivamente es usable.

Prueba de caja negra


```
=== INICIO DEL PROGRAMA ===
1. Cargando salas.txt...
2. Creando salas...
   [crearsala] Total salas creadas: 30
   numsal = 30
3. Cargando conexiones.txt...
4. Creando conexiones...
   [crearconex] Iniciando...
   [crearconex] Total conexiones creadas: 57
   numcon = 57
5. Cargando jugadores.txt...
6. Creando jugadores...
   Total jugadores: 0
   numjug = 0
7. Cargando objetos.txt...
8. Creando objetos...
   [crearobj] Total objetos creados: 10
   numobj = 10
9. Cargando puzles.txt...
10. Creando puzles...
   [crearpuz] Total puzles creados: 12
   numpuz = 12
11. Cargando partida.txt...
12. Creando partidas...
   [crearpar] Total partidas creadas: 1
   numpar = 1

13. Iniciando menú principal...

      --BIENVENIDO A ESI SCAPE--
      -----
1. Nueva partida.
2. Cargar partida.
3. Salir.
Elige una opcion: 
```

Plan de pruebas de aceptación

Para la aceptación del plan de pruebas de aceptación y que el usuario final de la aprobación a este nuestro trabajo, el sistema debería ser capaz de cumplir con el guión de trabajo propuesto (el cual cumple) y no presentar errores fatales, los cuales hemos solventado y demostrado en las anteriores pruebas, con lo cual es posible llegar a la aceptación de este sistema.

Documentación del código fuente

Durante la creación de este trabajo hemos colocado varios comentarios en el código fuente con la intención del fácil entendimiento de todo el código. Ya que son muchas líneas de código las que comprenden este trabajo, colocaremos algunos ejemplos para asegurar la rigurosidad en la documentación.

```
//Cabecera: void resolver(puzles *puz, partida p, int *numpuz, int *par)
//Precondición: La función recibe la estructura de puzles y partida inicializada
//Postcondición: El jugador podrá resolver puzles
void resolver(puzle *puz, partidas p, int *numpuz, int *par){
    char c[5];
    int x,y,resolver;
    for(x=0; x<=numpuz; x++){
        if(strcmp(p[*par].sala_actual, puz[x].id_sala)==0){           //Comprobamos que el puzle es
            printf("El puzle es de tipo %s\nDescripción: %s\n", puz[x].tipo, puz[x].desc);
            for(y=0; y<p.num_puzles; y++){
                if(p.puzles_estado[y].resuelto==true){             //Nos dice si el puzle está resuelto
                    printf("El puzle %s ya ha sido resuelto.\n", puz[x].nomb_puz);
                }
                else{
                    do{
                        printf("Puedes resolver el puzle %s, ¿quieres resolverlo? Si:1 No:0 \n", puz[x].nomb_puz);
                        scanf("%d", &resolver);
                    }while(resolver!=1 && resolver!=0);
                    if(resolver==1){
                        puz[x].sol;                                  //Muestra la solución del puzle
                    }
                }
            }
        }
    }
}
```